CUSTOM IC DESIGN, WITHOUT A CLOSED WORKFLOW

Give your design team another degree of freedom

Revolution EDA is an open, extensible environment for custom IC schematic, simulation, and layout workflows.

Executive briefing — 15 slides

The strategic risk is bigger than license cost

Locked

Data & workflows

Opaque formats and vendor-specific automation make change expensive.

Fragmented

Engineering flow

Design intent is handed between disconnected capture, simulation, and physical tools.

Constrained

Innovation

New methods wait for vendor roadmaps, scarce specialists, or brittle integrations.

> The question is not “Can we buy another tool?” It is “Can we preserve strategic control of how we design?”

The proposal: add optionality, not disruption

Start beside the incumbent flow

- Select a **bounded, non-tapeout-critical** design
- Connect one relevant **PDK and simulator path**
- Reproduce a known result
- Measure engineering effort and workflow fit
- Keep every pilot artifact portable

Earn expansion with evidence

No forced migration
No proprietary database commitment
No enterprise-wide rollout to approve

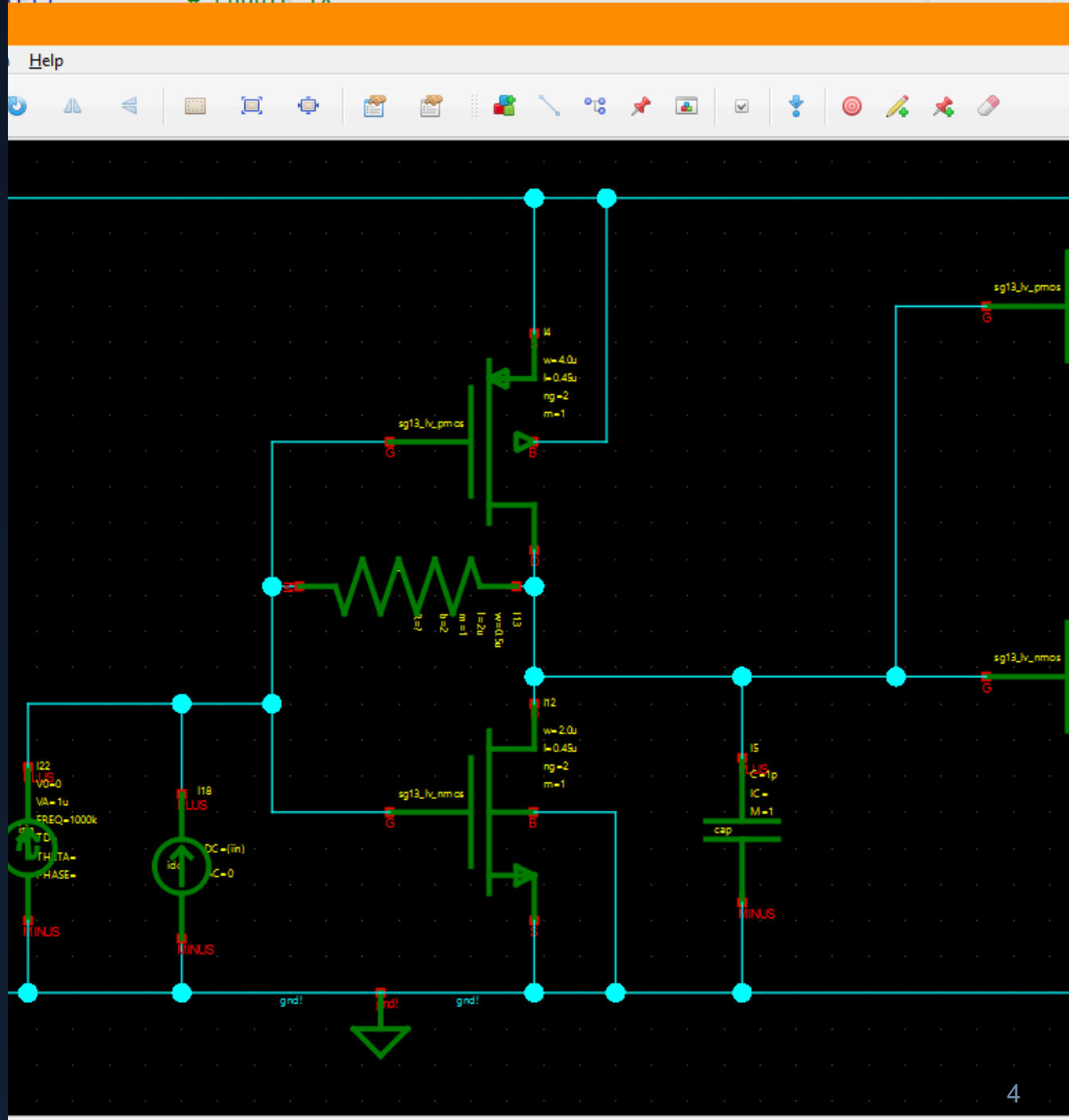
A pilot can answer the real question:

> Where does Revolution EDA create leverage for us now?

One environment across the custom IC loop

- **Schematic & symbol** editing
- Hierarchical, configuration-driven **netlisting**
- Optional **simulation and plotting** plugins
- Hierarchical **layout** with Python PCells
- **GDS and OAS** import/export
- RC parasitic extraction to **SPICE, SPEF, Spectre, and VACASK**
- Project, library, PDK, and plugin management

Current project version: 0.9.0. Optional capabilities depend on installed plugins, PDKs, and external simulators.



Design data your organization can inspect

Human-readable JSON

- Diffable in standard version-control systems
- Scriptable with familiar engineering tools
- Reviewable without a proprietary database reader
- Suitable for automation and reproducible transformations

```
{
  "cellView": "schematic",
  "viewType": "schematic"
}
```

> The design database becomes an accessible engineering asset—not a black box.

Integrate where standards already exist

Boundary	Revolution EDA path	Executive value
Process enablement	Python-based PDK modules and PCells	Adapt the environment to the process
Simulation	Xyce, Spectre, and VACASK netlisting paths	Preserve simulator choice
Model intake	Verilog-A and supported subcircuit parsing	Reuse existing model assets
Physical exchange	GDS / OAS import and export	Fit established downstream flows
Extracted output	SPICE / SPEF / Spectre / VACASK	Connect analysis and verification stages
Automation	Python console, JSON, plugin API	Build workflow IP with common skills

A platform, not a fixed appliance

PDK layer

Process layers, callbacks, labels, and PCells are supplied through Python packages.

Plugin layer

Plugins add menus, actions, and custom views without changing core application code.

Automation layer

An integrated Python console and transparent files expose practical extension points.

Build differentiated workflow IP at your pace

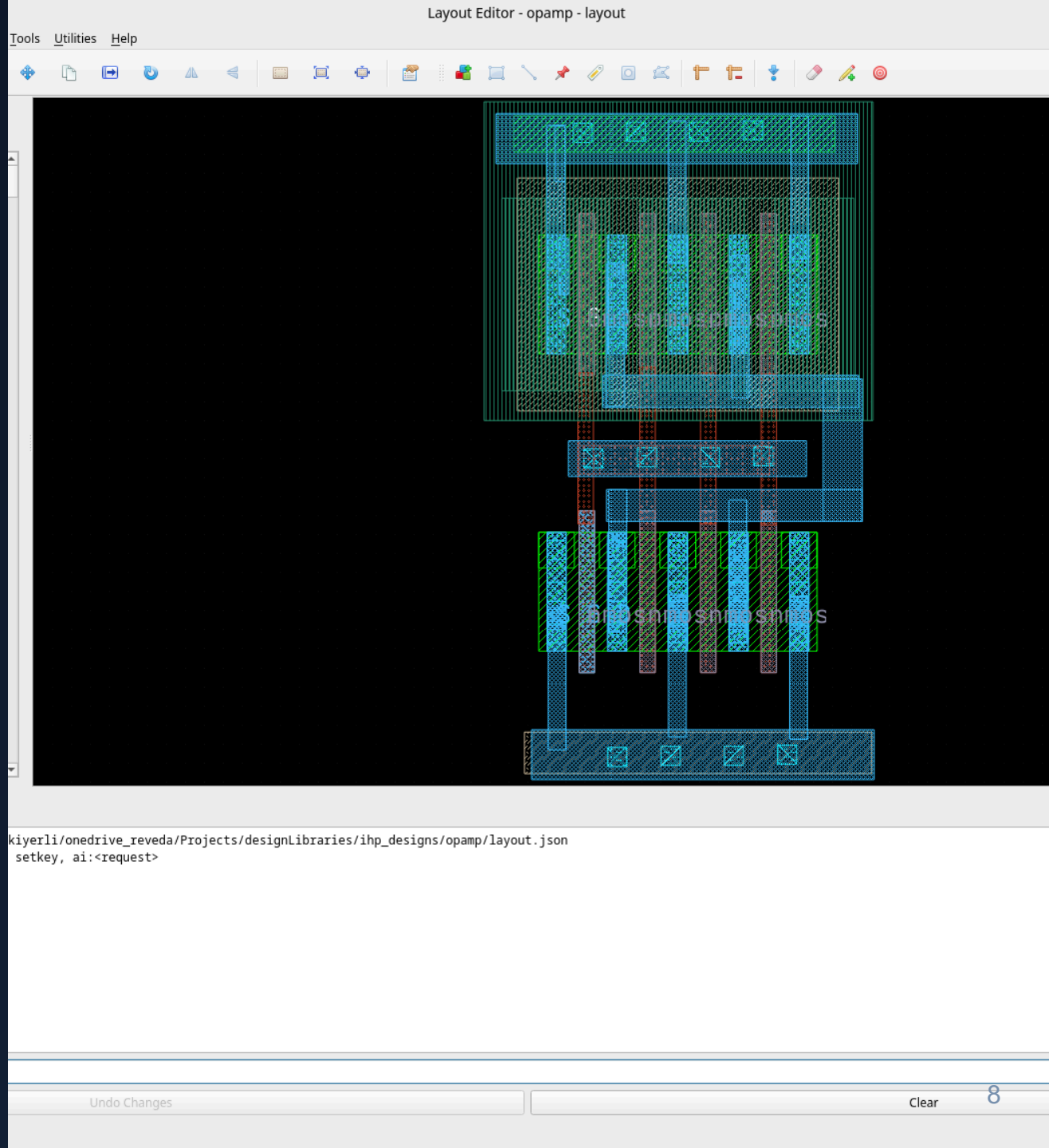
Source and binary plugin models allow **open collaboration** and **controlled commercial extensions** to coexist.

AI assistance—with a reversible operating model

- Natural-language editing of **schematics, symbols, and layouts**
- Choice of **OpenAI, Google, or Anthropic** backends
- A backup before each AI modification
- One-click restore / undo path
- Direct inspection of design JSON
- Encrypted local storage for API keys

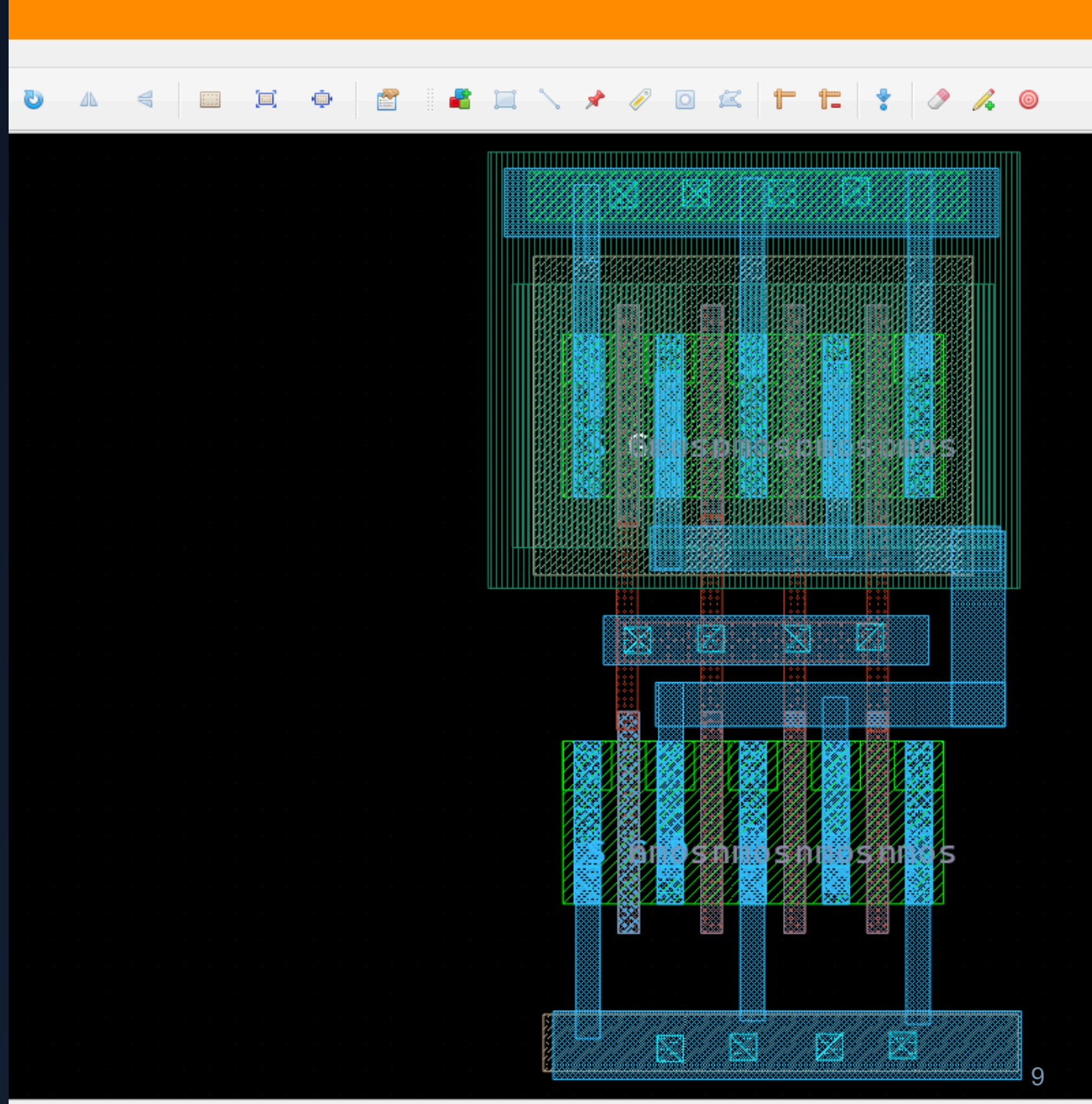
Treat AI as an accelerator—not an authority

Deployment should follow company policy for model access, design-data handling, review, and approval.



Physical design remains connected to intent

- Hierarchical custom layout
- Rectangles, paths, polygons, pins, labels, and vias
- Python-based parametric cells
- Layer visibility and selectability controls
- Initial schematic-driven layout support
- Connectivity-aware editing and net highlighting
- GDS / OAS exchange and RC extraction path



The business case is strategic leverage

For design leadership

Another path for prototyping, education, research, smaller programs, and workflow experiments.

For engineering

One inspectable environment with Python automation and fewer hard boundaries between design stages.

For the enterprise

Reduced dependence on any single interface, file format, deployment model, or vendor roadmap.

> Optionality compounds: every reusable PDK element, plugin, and automation script increases future choice.

Open core changes the control model

MPL 2.0 licensed core

- Use for any purpose
- Inspect and modify source
- Distribute under the license terms
- Build commercial services and derivatives

Practical implications

- Audit the implementation
- Preserve access to the core
- Qualify changes internally
- Keep plugins, PDKs, and libraries under their own applicable terms

This slide is an operational summary, not legal advice. Legal review should confirm obligations for the organization’s intended distribution model.

Address the adoption risks directly

Leadership concern	Pilot control
“Is it production mature?”	Version 0.9.0: evaluate on a bounded use case; do not assume tapeout readiness
“Will it disturb the current flow?”	Run side-by-side; exchange only through agreed files and interfaces
“Can it support our process?”	Make one target PDK path a pass/fail workstream
“Who supports it?”	Name an internal owner; evaluate documentation, issue response, and support options
“What about security and IP?”	Apply approved AI settings, dependency review, access controls, and artifact handling
“Will it scale?”	Test a representative hierarchy and dataset; record load, edit, netlist, and export behavior

A focused pilot can produce a decision

Suggested scope: one representative block, one PDK, one simulator path

1 — Establish

Install, register the PDK and library, import or recreate the reference design, capture baseline effort.

2 — Exercise

Edit hierarchy, netlist and simulate, make a layout change, exchange GDS/OAS, test one automation.

3 — Decide

Compare results, document gaps, estimate ownership needs, and choose stop / extend / adopt.

Deliverables: reproducible project • verified result comparison • issue log • workflow-fit scorecard • recommendation

Agree on success before the pilot starts

Dimension	Example evidence—not a preclaimed result
Correctness	Netlist and selected simulation outputs agree within an approved tolerance
Interoperability	Required model, GDS/OAS, and extracted-netlist paths complete without data loss
Productivity	Engineers complete defined edits and automation tasks; effort is recorded against baseline
Extensibility	One small plugin, script, PCell, or PDK adaptation is implemented and reviewed
Governance	Project is reproducible from controlled source, configuration, and dependency records
Adoption	Pilot users identify credible use cases and blocking gaps

Score the evidence. Do not grade the demo.

Authorize a controlled evaluation

Choose one sponsor, one engineering lead, one representative design, and one explicit decision date.

The ask

Approve the pilot scope and success criteria—not a migration.

The downside is bounded.
The learning is reusable.
The upside is strategic design freedom.